Auditoría del módulo de lotes y protocolos de engorde

Revisión iniciada el **20 de octubre de 2026** y cerrada el **21 de octubre de 2026**, America/Costa_Rica. Base inspeccionada: HEAD `dcb214c`; al comenzar, el único cambio local registrado era `Agents.md`. Instrucción aplicada: Agents.md, «Auditoría exhaustiva del módulo de lotes y protocolos de engorde».

Alcance: modelos, rutas, controladores, servicios, interfaz, tareas, alimentación, pesajes, sanidad, ventas, traslados, reportes, permisos y planes. Esta entrega documenta la auditoría y sus reproducciones; no modifica el código funcional ni remedia los hallazgos. Los valores productivos descritos son fórmulas del software, no recomendaciones zootécnicas.

1. Resumen ejecutivo

El manejo básico tiene controles reales, pero el conjunto todavía presenta fallas de aislamiento, integridad clínica, continuidad de operaciones e historia productiva. Las entradas y movimientos entre lotes, el cambio de potrero, los pesajes colectivos y el cierre del lote usan transacciones. Esto no se extiende a la creación completa del lote, el retiro manual, el cambio de etapa ni al conjunto de efectos de un protocolo de engorde.

Se documentan **21 hallazgos: 12 P1 y 9 P2**. P1 indica prioridad alta por seguridad, integridad o concurrencia; P2 agrupa analítica, experiencia de usuario y capacidades incompletas. No son puntuaciones CVSS. Algunos problemas atraviesan módulos ya auditados; se incluyen porque siguen siendo alcanzables desde este flujo.

Los principales resultados son:

- El alta del lote acepta `organizacionId` y `fincaId` del cliente. El plugin solo completa los valores ausentes; no reemplaza los inyectados.
- Las plantillas privadas de engorde se filtran por finca al listar, pero no al utilizarlas por ID ni al versionarlas. El límite organizacional permanece; falta el límite entre fincas de esa organización.
- El ejecutor sanitario reenvía campos internos de `datos.aplicacion` y permite seleccionar animales ajenos al lote dentro de la finca. Evita las validaciones del controlador sanitario ordinario.
- Si un paso falla después del pesaje/aplicación/cambio de ración, esos efectos pueden quedar confirmados aunque el ciclo no se guarde. La concurrencia optimista del ciclo no revierte efectos previamente confirmados.
- El cierre del lote termina pertenencias, rotación, plan y ración, pero deja tareas y ciclos de engorde vigentes. La salida de una pertenencia sobrescribe el evento de entrada en la bitácora individual.
- Un paso parcial completa su tarea y cuenta como realizado. Se puede repetir la misma etapa y finalizar un ciclo con actividades obligatorias pendientes.
- No se conservan pesos de entrada/salida en la pertenencia ni métricas finales del lote. El consolidado Premium utiliza un período de pesajes distinto al detalle.
- Hay consumo físico por lote, información comercial por animal y reportes financieros externos. **No hay costeo real integrado del ciclo ni conversión alimenticia calculada a partir de suministros y ganancia de peso.**

Controles comprobados

- `auth` exige usuario/membresía activos, organización activa y finca autorizada, y establece el contexto de consulta.
- `Lote`, `PertenenciaLote`, `EventoLote`, pesajes y ciclos tienen aislamiento por organización y finca en consultas normales.
- Existe un índice único parcial `{organizacionId, fincaId, animal}` para pertenencias con `activo: true`, además del código de lote único por finca.
- El ingreso valida animal activo, especie y objetivo productivo compatible; deduplica los IDs solicitados y rechaza pertenencias previas salvo movimiento explícito.
- Rotaciones tienen restricciones de unicidad; mover un lote valida el destino y su capacidad en ese momento. Se actualizan `Animal.loteActual` y `Animal.potreroActual` en los flujos principales.
- El pesaje grupal valida pesos, duplicados y pertenencia actual; registra `Pesaje`, peso actual y eventos en una misma sesión.
- Todo `/api/protocolos-engorde` exige `requireFeature('protocolosEngorde')`, que también verifica vigencia. Las tareas especializadas no pueden completarse por el ejecutor genérico de tareas.
- El ciclo guarda versión y snapshot de la plantilla; declara concurrencia optimista y un índice que impide dos ciclos `ACTIVO` del mismo lote.
- No existe ruta de borrado físico de lotes, plantillas o ciclos en los routers examinados.

Evidencia ejecutada y límites

Verificación	Resultado
`npm.cmd test` en backend	**272 aprobadas, 0 fallidas, 0 omitidas**, salida 0.
`node --test documentation/auditoria-lotes-engorde.repro.cjs` en backend	**18 reproducciones confirmadas**, salida 0. Comprueban defectos actuales, no su corrección.
`npm.cmd run test:lunar` en frontend	**6 aprobadas**, salida 0.
`npm.cmd run test:tareas` en frontend	**4 aprobadas**, salida 0.
`npm.cmd run build` en frontend	Compilación y PWA correctas. Paquete principal de **943,47 kB**, advertencia por superar 500 kB.

El primer intento de compilación falló porque el aislamiento del entorno impedía a esbuild leer un directorio antecesor. Se repitió con autorización de ejecución y finalizó correctamente; no fue un defecto del módulo.

Las reproducciones están en auditoria-lotes-engorde.repro.cjs. Ejecutan funciones/controladores reales con persistencia y efectos externos sustituidos por dobles; R1/R2/R3/R10 también usan validaciones o middleware reales de Mongoose. No abren conexiones a MongoDB. No prueban atomicidad real, despliegue de índices, tráfico HTTP autenticado, rendimiento bajo carga ni navegación de navegador. No se accedió a producción ni se enviaron avisos reales.

Mapa de implementación y APIs

Área	Implementación inspeccionada
Entidad y pertenencia	Lote.js, PertenenciaLote.js, HistorialEtapaLote.js, EventoLote.js, config/lotes.js.
Operación del lote	loteRoutes.js, lote-controller.js, lote-service.js.
Protocolos	PlantillaProtocoloEngorde.js, CicloEngorde.js, protocoloEngordeRoutes.js, controlador, servicio, scheduler. No existe un modelo principal denominado `ProtocoloEngorde.js`: la campaña es `CicloEngorde` y sus ejecuciones están embebidas.
Integraciones	alimentacion-service.js, planAlimentacion-service.js, pesaje-integridad-service.js, aplicacionSanitaria-service.js, eventoAnimal-service.js, ventaAnimalController.js, consolidacionMultiFinca-service.js.
Analítica	reporteLotes-controller.js, reporte-controller.js, indicesProductivos-service.js, reporteComprasAnimales-service.js.
Interfaz	Lotes.js, ProtocolosEngorde.js, Tareas.js, GestionConsolidacionPremium.js, api.js. Formulario y detalle están integrados en `Lotes.js`, no en archivos separados.

Rutas operativas bajo `/api/lotes`: `GET/POST /`, `GET/PUT /:id`, `POST /:id/animales`, `/mover-animales`, `/retirar-animales`, `/cerrar`, `/pesajes`, `/tareas`, `/cambiar-potrero`; `PATCH /:id/etapa`; consultas de historial, plan, ración y suministros; asignaciones `POST /:id/plan-alimentacion` y `/racion`.

Rutas bajo `/api/protocolos-engorde`: listar/crear/versionar plantillas, listar/crear/consultar ciclos, consultar actividad por tarea, ejecutar paso, avanzar etapa, finalizar, cancelar, candidatos de venta y `/metricas/consolidado`. Reportes relacionados: `/api/reportes/lotes`, `/lotes/analitica` y `/engorde`.

2. Catálogo exhaustivo de funcionalidades activas

«Básico» significa implementación operativa con alcance limitado; «Incompleto» identifica defectos materiales o integración ausente. No se certifica «100% Funcional» sin validar el conjunto con base de datos e interfaz reales.

Submódulo / Función	Descripción en código	Estado Backend	Estado Frontend	Madurez
Lote / identificación	Código, nombre, especie, propósito, descripción y creador.	Modelo e índice único por finca. Alta sin DTO estricto.	Formulario y listado real.	Incompleto
Lote / clasificación	Bovino/Porcino; ENGORDE, REPRODUCCION, REEMPLAZO, DESTETE,	Normalización y compatibilidad de objetivos.	Selectores y filtros.	Básico

Submódulo / Función	Descripción en código	Estado Backend	Estado Frontend	Madurez
	CUARENTENA, VENTA, OTRO.			
Lote / etapa	`etapaOperativa`, historial con fechas y actor.	Enum y cambio explícito; sin transacción integral.	Selector y cronología.	Incompleto
Lote / fechas y metas	`fechaInicio`, `fechaCierre`, `pesoObjetivoKg`, `gmdObjetivoKgDia`.	Persistencia; validación cronológica insuficiente.	Fecha civil y campos numéricos.	Incompleto
Lote / consulta	Lista, búsqueda, detalle, filtros de especie/propósito/etapa/estado/potrero.	Consultas reales; enriquecimiento por lote sin paginación.	Tabla con filtros.	Básico
Alta desde compra	Genera código por especie, propósito, año y consecutivo.	`crearLoteRapido`, puede compartir sesión con compra.	Compra grupal permite lote rápido.	Básico
Incorporación	Solo animales activos compatibles; pertenencia única.	Entrada transaccional, sincroniza lote/potrero del animal.	Selección de animales.	Básico
Movimiento	Cierra pertenencia previa y abre destino.	Transacción; valida destino en finca actual.	Acción Mover; la URL se utiliza como destino.	Básico
Retiro	Deja animales sin lote/potrero, con motivo e historial.	Secuencia sin transacción.	Acción Dejar sin lote con confirmación.	Incompleto
Historial poblacional	Entrada/salida, motivos, usuario y timestamps.	No almacena pesos de entrada/salida.	Historial de pertenencias.	Básico
Rotación	Traslada lote a potrero, cierra anterior y libera área.	Transacción, capacidad y unicidad al iniciar.	Formulario de traslado.	Básico
Cierre/cancelación del lote	Exige decidir destino de animales; cierra asignaciones y rotación.	Transacción, pero no cierra tareas/ciclo ni congela métricas.	Cierre deja animales sin lote; API también permite moverlos.	Incompleto
Pesaje colectivo	Registros individuales reales con lote y eventos.	Sesión para pesajes, pesos actuales y bitácora.	Captura colectiva en lote y en protocolo.	Básico
Resumen de pesos	Último peso, cobertura, peso cercano a entrada y metas.	Datos calculados desde pesajes.	Tarjetas, tablas y reportes.	Incompleto
GMD del lote	Ganancia dividida entre suma de días observados por animal.	Usa pertenencias activas y pesajes desde su entrada.	Visible desde Gestión.	Básico
Sanidad / resumen	Estados actuales y tratamientos activos de integrantes.	Consulta, sin inventar aplicaciones.	Panel y acceso a sanidad.	Básico
Plan alimentario	Estrategia de manejo e historial de asignaciones.	Plan activo compatible y asignación única.	Selección desde detalle.	Básico
Ración	Composición prevista, base por animal/lote, historial.	Asignación transaccional y compatibilidad.	Detalle y cambio desde protocolo/alimentación.	Básico
Suministros	Kg entregados, sobrante, población y snapshots.	Registro real separado del protocolo.	Enlace a Alimentación y último suministro.	Básico
Tarea grupal manual	Tarea única referenciada al lote.	DTO y responsable validados; evento posterior separado.	Programar tarea.	Básico
Bitácoras	Eventos operativos del lote e individuales.	Evento de salida reemplaza entrada individual; lote limita a 100.	Cronologías reales.	Incompleto

Submódulo / Función	Descripción en código	Estado Backend	Estado Frontend	Madurez
Plantillas	Etapas, acciones, objetivos y criterios de salida.	Alta, versión y ámbito finca/organización.	Editor integrado.	Incompleto
Ciclo de engorde	Snapshot de protocolo y participantes iniciales, responsable y etapas.	Solo lote bovino de engorde activo, con integrantes.	Inicio y operación del ciclo activo.	Incompleto
Cronograma	Desplazamientos horarios desde inicio o paso.	Genera ejecuciones y tareas; no resuelve bien ciclos/dependencias futuras.	Editor de referencias y horas.	Incompleto
Paso PESAJE	Crea mediciones de animales actuales.	Ejecuta servicio real; no asegura toda la población ni atomicidad con ciclo.	Pesos individuales; permite dejar filas vacías.	Incompleto
Paso CAMBIO_RACION	Cambia asignación de ración.	Efecto real; no registra suministro ni consumo.	Selector de raciones.	Básico
Paso SANIDAD	Aplicación única con animales, producto y retiro de carne si se envía.	Servicio real; falta DTO/validación del alcance del paso.	Producto, tipo, dosis y vía; sin captura de días de retiro en estos formularios.	Incompleto
Otros siete tipos de paso	RECEPCION, IDENTIFICACION, UBICACION, REVISION_SANITARIA, CONTROL, EVALUAR_VENTA, OTRA.	Marcan ejecución/resultado y tarea. No ejecutan alta, traslado o venta por sí solos.	Fecha y observaciones.	Básico
Avance de etapa	Cierra intervalo y genera cronograma de otra etapa.	No exige obligatorios completos; permite volver/repetir por API.	Avanzar etapa sin criterios bloqueantes.	Incompleto
Finalizar/cancelar ciclo	Estado y fecha final, cancelación de pendientes y evento.	No finaliza lote; secuencia no atómica.	Botones de cierre.	Incompleto
Candidatos de venta	Último peso frente a meta del lote o etapa.	Lista animales; no registra la venta.	Abre Ventas con selección.	Básico
Cumplimiento	Proporción de obligatorios realizados y a tiempo.	PARCIAL cuenta como realizado completo.	Porcentaje y estados.	Incompleto
IEE	Componentes productivos por especie y período.	Implementado; no es conversión alimenticia ni índice exclusivo de un lote.	Reportes productivos.	Básico
Consolidado Premium	Lotes activos, peso/GMD, días, venta y cumplimiento por finca.	Implementado; usa todo el historial de peso de miembros actuales.	Panel Premium.	Incompleto
Costeo del engorde	Compra y venta externas; no costo real del ciclo.	No suma costos de suministros ni calcula costo/kg ganado por lote.	Sin cuenta económica integral del lote.	Incompleto
Offline específico	Operación de lotes/ciclos mediante API.	Sin protocolo de sincronización/idempotencia integral de estos comandos.	No se encontró cola específica para lotes/ciclos; tareas genéricas offline no sustituyen ejecución especializada.	Incompleto

3. Ciclo de vida del lote: creación → asignación → rotación → cierre

3.1 Esquemas y pertenencia única

`Lote` usa `etapaOperativa`, `ubicacionActual`, `fechaInicio` y `gmdObjetivoKgDia`; no `etapaActual`, `potreroActual`, `fechaCreacion` o `metaGmd` como nombres principales. Estados: `ACTIVO`, `CERRADO`, `CANCELADO`. `createdAt` es la fecha técnica de creación, distinta del inicio productivo.

`PertenenciaLote` contiene `animal`, `lote`, `fechaEntrada`, `fechaSalida`, motivos, `activo` y `registradoPor`. **No tiene `pesoEntrada` ni `pesoSalida`.** Su índice único parcial garantiza, si está desplegado, una pertenencia activa por animal dentro de la misma organización/finca. No es un índice global por animal; la coherencia entre fincas depende también de los comandos de traslado y de impedir escrituras con tenant incorrecto.

El detalle usa pertenencias activas como población; la rotación actualiza animales usando `Animal.loteActual`. Si esas dos representaciones divergen por una operación incompleta, distintas pantallas pueden operar sobre conjuntos diferentes.

3.2 Creación y asignación

El controlador de alta guarda lote, historial inicial y evento antes de incorporar animales y moverlo al potrero. Las operaciones internas pueden tener transacción, pero no comparten una sesión que abarque toda el alta. La interfaz ofrece potrero inicial sin permitir incorporar animales en ese formulario: el traslado se rechaza por lote vacío después de guardar el lote. Además, «Sin etapa» se envía como `''`, rechazado por el enum.

Al incorporar animales se comprueba el lote activo, existencia de todos los IDs, especie, estado y objetivo. Se crea pertenencia y se actualizan `loteActual` y `potreroActual`. No se captura peso de entrada: el resumen busca posteriormente el pesaje más cercano, incluso posterior a la entrada, dentro de ±30 días.

Las consultas de lote y animales usan el contexto autenticado: un ID ordinario de otra finca no debería resolverse en esta operación. Esto no subsana el defecto de tenant del alta ni el de las plantillas.

3.3 Movimiento, retiro, venta y traslado de finca

`mover-animales` determina el destino mediante `body.loteDestino || params.id`; la interfaz envía el destino en ambos lugares. No es una ruta que verifique un lote de origen declarado en la URL. El servicio localiza la pertenencia activa real y la cierra. Origen y destino quedan en la misma sesión, junto con el animal y eventos. El potrero del animal pasa al del destino o queda nulo.

El retiro manual cierra pertenencias una a una y limpia lote/potrero del animal sin una sesión común. Usa motivo libre y no registra un peso de salida ni convierte por sí mismo el estado del animal a Vendido/Muerto.

La venta confirmada llama `cerrarPertenencia` con sesión y motivo `VENTA`. La baja por estado del animal y el traslado entre fincas también cierran pertenencias; el traslado de finca limpia lote y potrero en su transacción. **Revertir la venta reactiva la pertenencia anterior sin verificar que el lote siga activo, ni restaurar el potrero actual del lote** (LE-10).

La rotación conserva `numeroAnimales` como snapshot de inicio, según el diseño documentado. Entradas posteriores no vuelven a comprobar capacidad y salidas totales no terminan automáticamente esa rotación. No debe interpretarse ese snapshot como censo vivo.

3.4 Cambio de potrero y cierre

Mover el lote exige lote activo, destino válido para pastoreo, al menos un integrante, capacidad suficiente y ausencia de otra rotación activa. Cierra la anterior, pasa su potrero a `Descanso` salvo estado Mantenimiento/Inactivo, abre la nueva y sincroniza ubicación/animales dentro de una sesión.

El cierre **no exige lote vacío**, pero con integrantes exige `SIN_LOTE` o `MOVER` y destino diferente. En la misma transacción:

1. Cierra pertenencias y limpia animales, o los mueve al destino.
2. Guarda `CERRADO`/`CANCELADO` y `fechaCierre`.
3. Finaliza la rotación y libera su potrero a `Descanso` si corresponde.
4. Limpia `ubicacionActual`.
5. Desactiva `AsignacionPlanAlimentacion` y `AsignacionRacionLote`, con fecha de fin.
6. Cierra intervalos de etapa y registra `LOTE_CERRADO`.

No cancela tareas, no finaliza/cancela `CicloEngorde` y no calcula ni congela métricas finales. Solo valida la fecha contra la entrada de una rotación existente; no contra todos los intervalos de pertenencia, el inicio del lote o la fecha actual. Se puede volver a cerrar y alterar su fecha. Crear tareas desde lote tampoco exige que este siga activo.

4. Evaluación de protocolos de engorde

4.1 Plantilla, campaña y ejecución

La plantilla es exclusivamente bovina (`BOVINO`). Tiene etapas ordenadas, metas, criterios TIEMPO/PESO_PROMEDIO/GMD/EVENTO/MANUAL y pasos con offsets, ventanas, obligatoriedad, rol y configuración. Los criterios **sugieren** avance; no son una máquina automática de fases por peso.

El ciclo necesita lote `Bovino`, `ENGORDE`, `ACTIVO`, integrantes y responsable con membresía vigente y acceso a la finca. Guarda snapshot de plantilla y participantes iniciales (`pesoInicialKg` sale de `Animal.pesoActual`, no de un pesaje congelado validado al inicio). Los ejecutores y métricas utilizan la población actual del lote, no necesariamente la población de ese snapshot.

Se generan tareas para todos los pasos de la etapa que tengan `generaTarea`. Todas se asignan al responsable del ciclo: `rolResponsable` **no decide el destinatario**. La clave automática es ciclo+paso, sin identificar una nueva visita a la etapa; repetir una etapa puede reutilizar tareas anteriores.

Acción	Efecto que realmente ejecuta
PESAJE	`registrarPesajesLote`: crea Pesaje real, actualiza peso y bitácora. Acepta un subconjunto de integrantes con peso informado.
CAMBIO_RACION	`asignarRacionLote`: termina asignación anterior y crea la nueva. No entrega alimento ni consume existencias.
SANIDAD	`crearAplicacionSanitaria`: aplicación única real, eventos y retiro de carne si fue capturado. El ejecutor no usa el DTO sanitario del controlador ni exige pertenencia de cada seleccionado al lote.
RECEPCION / IDENTIFICACION	Registro de actividad; no crea/identifica animales ni los ingresa al lote automáticamente.
UBICACION	Registro de actividad; no llama a `moverLoteAPotrero`.
REVISION_SANITARIA / CONTROL	Registro de resultado/observaciones; no aplicación médica automática.
EVALUAR_VENTA / OTRA	Registro de actividad; no registra venta. Los candidatos se obtienen por otro endpoint.

No hay un ejecutor propio de implantes ni de desparasitación distinto de la captura sanitaria genérica. No se encontró consumo de inventario de medicamentos en este servicio. Los formularios SANIDAD de Protocolos y Tareas no solicitan días de retiro de carne; el servicio usa cero cuando faltan. Es una limitación de datos, no una afirmación de que todos los productos requieran el mismo retiro.

Los servicios de pesajes, ración y aplicación sanitaria pueden ser transaccionales internamente, pero `ejecutarPaso` los llama sin compartir una sesión con tarea/ciclo/evento. No hay clave idempotente integral del paso. PARCIAL permite volver a ejecutar, sobrescribiendo el resultado anterior y sin llevar un registro acumulado de participantes pendientes.

4.2 Fórmulas y límites de interpretación

Detalle de lote: para cada integrante actual, toma pesajes desde `fechaEntrada`, calcula ganancia entre primero y último y días entre ambos. `GMD lote = Σ ganancia kg / Σ días observados por animal`. Requiere al menos dos pesajes válidos y días positivos. `Cumplimiento GMD = GMD / gmdObjetivoKgDia × 100`. El peso actual es promedio de últimos pesajes disponibles, sin umbral de antigüedad; se devuelve cobertura. No hay filtro superior de fecha de cierre en esta consulta.

Consolidado Premium: también divide ganancia total por animal-días, pero usa primer/último pesaje de toda la historia accesible de cada integrante actual, sin recortar por entrada o inicio de ciclo. R16 reproduce **1 kg/día en detalle y 5,5 kg/día en consolidado** para los mismos animales y fechas. Suma ciclos `ACTIVO`, no todos los lotes operativos ni ciclos finalizados.

Cumplimiento de pasos: `realizados / obligatorios × 100`; tanto REALIZADO como PARCIAL cuentan completos. El porcentaje a tiempo usa ventana o inicio/fin de día del servidor. Las actividades obligatorias aún no creadas de etapas futuras no están en el denominador.

`/api/reportes/engorde`: selecciona animales con objetivo ENGORDE por especie y período; no recibe un lote/ciclo como cohorte exclusiva. Usa metas de configuración productiva, no las particulares de cada lote. Calcula GMD ponderada por animal-días, cumplimiento, eficiencia temporal, supervivencia, días y peso proyectados.

- Ganancia: último menos primer peso válido del período.
- Días objetivo: `(peso objetivo − peso inicial, mínimo 0) / GMD objetivo`.
- Eficiencia temporal: días objetivo / días reales o proyectados × 100. Para un animal vendido/muerto, usa los días observados como finalizados.
- Supervivencia: `(animales considerados en el período − muertes fechadas en período) / animales considerados × 100`. Expone muertes sin fecha confiable por separado; el denominador no es el inventario congelado del inicio de un lote.
- IEE: media ponderada de cumplimiento GMD **60 %**, eficiencia temporal **25 %** y supervivencia **15 %**, con techo 100 en cada componente y renormalización sobre componentes disponibles. Exige cumplimiento GMD finito para publicar IEE; no aplica un piso cero a cada componente.

`preparadoPara: ['conversionAlimenticia', 'eficienciaEconomica']` es una etiqueta de salida. **No ejecuta esos cálculos.** No hay en este reporte una división de kg de alimento consumido entre kg de ganancia. Tampoco se comprobó una conversión alimenticia teórica publicada por el módulo: simplemente falta esa capacidad.

5. Realidad de la integración con costos

La integración es **parcial**, no totalmente desconectada de Finanzas:

Dato/cálculo	Evidencia y alcance real
Compra inicial	`CompraAnimal` guarda líneas, peso, precio, ajustes y `loteAsignado`; el animal conserva `compraId`. Compra/venta generan movimientos financieros externos. El detalle del lote solo cuenta origen comprado/no comprado, no suma inversión inicial.
Margen comercial por animal	`reporteComprasAnimales-service.js:178` calcula `montoVenta − montoAsignadoCompra` para vendidos. No descuenta alimentación/sanidad del ciclo ni produce una cuenta económica del lote.
Gasto operativo estimado	`reporte-controller.js:742` reparte gastos del período por animales y meses y los aplica a duración de estancia. Es un reparto financiero externo; no una imputación de suministros al ciclo. Mantiene monedas separadas.
Suministro de alimentación	Guarda cantidades físicas, sobrante y snapshots. No contiene precio/costo unitario ni costo total histórico. Las referencias de origen COMPRA/CORTE_FORRAJE no equivalen a un cálculo de costo.
Reparto entre animales	`kgSuministradosPorCabeza` y consumo estimado por cabeza dividen cantidades físicas entre población capturada. No reparten importes monetarios.
Costos sanitarios del protocolo	No se encontró acumulación económica por etapa, ciclo o animal en el ejecutor de engorde.
Costo/kg ganado y margen real del lote	No implementados en lote/ciclo/reporte de engorde. Tampoco hay cierre económico congelado del ciclo.

Por tanto, no debe presentarse GMD, IEE, consumo por cabeza o margen comercial como rentabilidad real de engorde. LE-19 propone desarrollar esa integración como capacidad de producto, no corregir una fórmula monetaria existente que aquí no existe.

6. Cumplimiento de exclusividad por plan y roles

6.1 Planes

Los precios de referencia de `Agents.md` (10/20/35/60 USD) **no coinciden con los valores predeterminados del repositorio**. `config/planes.js` declara 6/12/20/35 USD y permite sobrescribirlos mediante variables de entorno. No se inspeccionaron secretos ni se verificaron precios efectivos en Stripe/producción; esto es una discrepancia documental a conciliar, no prueba de cobros incorrectos.

Plan	Capacidad y controles reales
ESENCIAL	Lotes operativos y alimentación permitidos; una especie. GET de lista/detalle oculta GMD/metas avanzadas con `proyectarDetalleLotePorPlan`. Varias respuestas de escritura devuelven detalle completo sin proyección (LE-12).
GESTION	`analiticaProductiva` habilitada; `/reportes/lotes/analitica` y `/reportes/engorde` la exigen. Protocolos bloqueados. No se encontró una curva específica por lote en `Lotes.js`; hay indicadores y tablas.

Plan	Capacidad y controles reales
PRO	`protocolosEngorde` habilitada y exigida por todo el router; vigencia incluida. La versión privada de plantilla no está aislada por finca al buscar por ID.
PREMIUM	Hereda Pro y habilita `reportesMultiFinca`: consolidado de engorde y creación de plantillas organizacionales. Hay interfaz Premium real.

`requireFeature` verifica vigencia, aunque la ruta no agregue además `requirePlanVigente`. No debe declararse ausente la protección de protocolos por ese motivo. En lotes, alta/PUT verifican vigencia indirectamente por `asegurarPuedeUsarEspecie`; plan/ración/tarea tienen middleware explícito. **Entradas, movimientos, retiro, cierre, etapa, pesajes y cambio de potrero carecen de ese control de vigencia**. `auth` deja pasar una organización activa con plan suspendido; `req.planVigente` es informativo y no bloquea por sí solo.

El consolidado limita a `req.usuario.fincas` cuando la membresía es restringida y omite el filtro de finca solo para consultar varias autorizadas; el plugin conserva la organización. El servicio interpreta lista vacía como todas: sería más robusto distinguir explícitamente acceso global/lista vacía, pero no se confirmó un exploit de ese caso porque `auth` resuelve antes una finca autorizada. No se cuenta como hallazgo confirmado independiente.

6.2 Roles

Operación	Backend
Ver lotes	Administrador, Encargado, Trabajador, Veterinario, Consulta.
Crear, editar, mover miembros, retirar, cerrar, cambiar etapa	Administrador y Encargado. Trabajador no administra movimientos.
Pesajes de lote por ruta directa	Administrador, Encargado y Veterinario.
Potrero y programación de tareas	Administrador y Encargado según permisos específicos.
Configurar/versionar protocolo	Administrador.
Iniciar, avanzar y cerrar ciclo	Administrador y Encargado.
Ejecutar paso	Administrador/Encargado; Veterinario libre para SANIDAD/REVISION_SANITARIA; otros casos requieren tarea asignada y rol permitido por la ruta. Trabajador puede ejecutar pasos asignados.

La interfaz reutiliza `soloLectura` del inventario para múltiples operaciones. Esto no representa todos los permisos anteriores: Encargado ve configuración reservada al Administrador; Veterinario puede tener pesajes permitidos por API pero acciones ocultas en lotes; los responsables de pasos no se resuelven por `rolResponsable`. La ruta de tareas especializadas permite al trabajador ejecutar lo asignado, pero no corrige estos desajustes del panel de lotes.

7. Plan de acción priorizado

Los hallazgos siguientes separan resultados reproducidos (R1–R18) de conclusiones por inspección. Las prioridades no significan que se haya verificado explotación en producción.

P1 — Seguridad, integridad y concurrencia

LE-01 — Alta de lote permite inyectar organización/finca y campos internos

Evidencia: lote-controller.js:57, organizacion-plugin.js:109, R1. El controlador extrae ubicación/animales y propaga el resto a `Lote.create`; `pre('validate')` asigna tenant únicamente si falta. Se aceptan también estado y fecha de cierre sin pasar por el comando formal.

Impacto: un gestor de una finca puede intentar insertar lotes en otra finca u organización con IDs conocidos. El documento puede guardarse y luego fallar su consulta desde la finca actual, devolviendo error después de la escritura. El evento usa el contexto actual, por lo que puede quedar una referencia cruzada. No se afirma ejecución de operadores `$set` en `create`: el defecto demostrado es asignación masiva de campos válidos del esquema.

Acción: DTO explícito; rechazar tenant/campos internos y operadores; imponer tenant y estado inicial desde servidor. Validar todo antes de guardar y ejecutar alta completa en una sesión. Probar el controlador con un ID externo y comprobar ausencia de

documentos/eventos persistidos.

LE-02 — Acceso a plantillas privadas de otra finca por ID

Evidencia: protocoloEngorde-service.js:39, `versionarPlantilla:57`, `crearCiclo:100`; modelo de plantilla. R3 confirma que el middleware de consulta añade organización pero no finca; R15 versiona una plantilla B desde contexto A y desactiva la anterior.

Impacto: el listado oculta plantillas ajenas, pero iniciar un ciclo con su ID permite reutilizarlas y versionar permite modificar su vigencia. `versionarPlantilla` conserva `fincaId` del documento anterior al cambiar alcance, pudiendo dejar alcance y finca incoherentes. Tras una bajada de Premium a Pro, las plantillas ORGANIZACION siguen visibles/utilizables: el gate de Premium solo se verifica al crear/versionar ese alcance.

Acción: resolver toda plantilla mediante filtro de organización y `(finca actual OR ámbito organizacional autorizado por plan)`. No confiar en el listado. Normalizar finca según alcance; versionar/desactivar atómicamente, con control de versión y unicidad lógica. Probar actor limitado a finca A contra ID privado de B.

LE-03 — El paso SANIDAD permite campos protegidos y animales fuera del lote

Evidencia: protocoloEngorde-service.js:176, aplicacionSanitaria-service.js:178, aplicacionSanitaria-controller.js:59, R10. El flujo ordinario usa `sanitizarAplicacionSanitaria`; el protocolo propaga `...datos.aplicacion` directamente. El servicio valida especie/estado y existencia del lote, pero no que cada animal pertenezca a él.

Impacto: un ejecutor asignado puede reenviar tenant/atributos internos o aplicar a otros bovinos activos de la finca, dejando la operación vinculada a un lote distinto. El esquema real acepta el tenant proporcionado. `responsableUsuario` no se contrasta aquí con membresía vigente. Una `claveIdempotencia` suministrada dentro de aplicación puede recuperar una aplicación previa sin verificar que corresponda a esta ejecución. Los formularios no permiten registrar días de retiro de carne, que quedan en cero si faltan.

Acción: DTO sanitario común al servicio y al controlador, tenant impuesto, responsable validado y participantes limitados al alcance de la ejecución. Generar idempotencia desde ciclo/paso/participante, no desde un valor arbitrario del objeto sanitario. Capturar y conservar retiro de carne cuando corresponda. Verificar rollback ante tenant/animal ajeno y que un replay no certifique una aplicación de otro paso.

LE-04 — El protocolo confirma efectos antes de confirmar su ejecución

Evidencia: protocoloEngorde-service.js:100, `ejecutarPaso:162`, `avanzarEtapa:187`, `cerrarCiclo:205`; R4. En ejecución se confirma el servicio de negocio, se actualiza tarea y luego se guarda ciclo y evento. Creación, avance y cierre también tienen escrituras separadas.

Impacto: un fallo deja pesaje/aplicación/ración o tarea confirmados sin ejecución consistente. Reintentar puede duplicar efectos. Dos ejecutores pueden confirmar efectos externos antes de que la concurrencia optimista rechace uno de los `ciclo.save()`. Un cierre que guarda el ciclo y falla al cancelar tareas no se repara al repetir: la respuesta temprana devuelve el ciclo ya finalizado.

Acción: sesión común y propagada a todos los servicios; escritura condicionada por versión/estado; clave idempotente del comando y resultado verificable. Avisos después del commit. Pruebas con replica set, dos solicitudes simultáneas y fallo en cada punto de persistencia. No asumir que transacciones internas independientes equivalen a una transacción del protocolo.

LE-05 — Alta, retiro y cambio de etapa del lote no son atómicos

Evidencia: lote-controller.js:57, lote-service.js:136, `retirarAnimalesDelLote:356`, `crearTareaLote:447`; R7 y R17.

Impacto: retirar puede guardar pertenencia inactiva antes de fallar al limpiar el animal. Crear lote puede devolver error conservando el lote creado. Cambiar etapa cierra el historial anterior antes de validar/guardar la nueva etapa; una etapa inválida puede dejar el historial cerrado sin el cambio. El PUT guarda campos antes de invocar ese cambio. Crear tarea puede fallar al registrar evento dejando la tarea creada.

Acción: validar previamente y compartir sesión en cada comando completo. Usar escritura condicionada para evitar cambios concurrentes incompatibles. Probar fallo en animal, historial y evento; verificar que no quede un subconjunto de cambios.

LE-06 — Cerrar el lote deja ciclo de engorde y tareas operativos

Evidencia: lote-service.js:278, `crearTareaLote:447`, protocoloEngorde-service.js:162; R13.

Impacto: lote cerrado y sin animales puede conservar ciclo ACTIVO y tareas pendientes. Los pasos genéricos solo verifican ciclo activo; pueden ejecutarse después del cierre del lote. Pesaje/ración encuentran otras restricciones y fallan, produciendo comportamiento desigual. Es posible crear nuevas tareas del lote cerrado. Finalizar el ciclo tampoco cierra por sí mismo el lote: son comandos diferentes sin política de coordinación explícita.

Acción: definir una política de cierre que resuelva o bloquee ciclos y tareas pendientes, aplicarla en la misma transacción y comprobar estado del lote en los ejecutores. Preservar tareas terminadas. Añadir aceptación: tras cierre no existe operación pendiente ejecutable sobre ese lote.

LE-07 — Máquina de estados y cobertura del paso no reflejan trabajo completo

Evidencia: protocoloEngorde-service.js:162, `avanzarEtapa:187`, `cerrarCiclo:205`; protocoloScheduler-service.js:66. R5/R6.

Impacto: PARCIAL completa la tarea y cuenta 100 %; el frontend solo ofrece ejecutar PENDIENTE. Un pesaje de un único integrante puede marcar toda la actividad REALIZADA. El servicio no contrasta población esperada/realizada; `ejecucion.animales` puede incluso quedar vacío porque el formulario envía IDs dentro de pesajes. Se puede repetir/retroceder etapa por API, generar ejecuciones repetidas y reutilizar claves de tareas. Finalizar cancela pendientes obligatorios sin exigir justificación. Una tarea cancelada puede completarse desde protocolo porque se comprueba asignado, no su estado.

Acción: transición explícita y condicionada; registro acumulativo de participantes y efectos, tarea en proceso mientras falten integrantes, pendientes visibles y ejecutables en interfaz. Un avance manual excepcional debe requerir motivo y registrar qué se omitió. Si se admite reingreso a una etapa, identificar cada visita con su propio ID/clave; de lo contrario, bloquearla.

LE-08 — Suscripción suspendida permite mutaciones de lotes

Evidencia: loteRoutes.js, auth.js:136, plan-service.js:178; R12 recorre el stack real de rutas con dobles de autorización/plan.

Impacto: entradas, movimientos, retiros, cierre, etapa, pesajes y cambio de potrero no pasan por validación de vigencia. Un plan suspendido con organización aún Activa puede alcanzar esos controladores. Alta/PUT, asignación alimentaria y programación de tareas sí tienen controles directos o indirectos; protocolos están protegidos por feature+vigencia.

Acción: middleware uniforme para toda mutación de lotes, manteniendo los permisos específicos. Prueba HTTP con suscripción Suspendido/Cancelado en cada ruta y comprobación de cero escrituras.

LE-09 — Fechas de pertenencia, etapa y cierre admiten historia incoherente

Evidencia: PertenenciaLote.js, lote-service.js:168, `agregarAnimalesAlLote:194`, `cerrarLote:278`, `cambiarEtapa:136`; R13 demuestra que un cierre ya realizado puede recibir una fecha anterior en otro cierre.

Impacto: no se rechaza sistemáticamente salida anterior a entrada, entrada anterior al lote, cierre anterior a pertenencias o fechas futuras que cambian el estado actual inmediatamente. El PUT permite cambiar el inicio aun con historia existente. El evento individual de ingreso no recibe `datos.fechaEntrada` y usa hora del servidor; el evento operativo de incorporación tampoco recibe esa fecha. Los cálculos e historial pueden contradecirse.

Acción: reglas temporales de dominio, distinción fecha civil/instante, fecha efectiva única propagada a eventos, comando explícito de corrección histórica y cierre idempotente. Probar cronología con/sin rotación; la comprobación actual frente a la rotación no cubre las demás relaciones.

LE-10 — Revertir venta puede reactivar pertenencia de un lote cerrado

Evidencia estática: ventaAnimalController.js:338, `restaurarPertenenciasCerradasPorVenta`. Busca pertenencias cerradas por VENTA, puebla solo código/nombre de lote, reactiva y asigna `loteActual`. No valida estado ni ubicación del lote.

Escenario: vender animales, cerrar su lote, anular/revertir venta. Se restaura una pertenencia activa al lote cerrado. Si el lote sigue activo pero cambió de potrero, tampoco se sincroniza `potreroActual` al restaurar. El snapshot de participantes del ciclo no resuelve esta inconsistencia.

Acción: al revertir, dejar sin lote o exigir destino activo compatible; si se restaura, validar estado y sincronizar potrero dentro de la transacción. Probar venta→cierre→anulación y venta→cambio de potrero→anulación. No se ejecutó esta secuencia contra MongoDB en esta auditoría.

LE-11 — La salida sobrescribe el evento individual de entrada

Evidencia: lote-service.js:122, `cerrarPertenencia:168`; eventoAnimal-service.js:3; R18.

Impacto: entrada y salida usan el mismo animal, módulo Lotes, referencia de pertenencia y tipo `Cambio de lote`. El upsert usa esos cuatro campos, sin distinguir acción. Al salir desaparece el evento original de entrada de la bitácora individual, aunque la pertenencia conserve sus fechas. Revertir venta vuelve a actualizar esa misma identidad.

Acción: identificar eventos por acción/ocurrencia además de pertenencia, manteniendo idempotencia por comando. Reconstruir historia solo con evidencia de pertenencias y operaciones, sin inventar fechas o actores. Prueba: ingreso, salida y reversión deben ser eventos distintos.

LE-21 — La capacidad del potrero se puede superar agregando animales después

Evidencia estática: lote-service.js:194 frente a `moverLoteAPotrero:494`. El segundo comprueba capacidad; el primero asigna automáticamente `potreroActual` del lote sin consultar capacidad/estado actuales del potrero.

Escenario: iniciar rotación de 5 animales en potrero de capacidad 5 y agregar/mover otros 3 animales al mismo lote. La validación de especie/objetivo no impide esa incorporación. Tras retirar/vender todos, la rotación puede seguir Activa y el potrero Ocupado hasta un cierre explícito.

Acción: comprobar capacidad y destino también en cambios de población, con un mecanismo que evite carreras. Definir cómo tratar rotaciones que quedan vacías. Mantener `numeroAnimales` como snapshot, si ese es el contrato, pero exponer ocupación actual por separado; no sobrescribir silenciosamente la fotografía histórica.

P2 — Analítica, capacidades y experiencia de usuario

LE-12 — Esencial recibe analítica avanzada en respuestas de escritura

Evidencia: lote-controller.js, respuestas de crear/actualizar/agregar/mover/retirar/etapa/potrero; reporteLotes-controller.js:43. R11 devuelve `gmdPromedioLote` con feature falsa.

Impacto: GET de detalle/lista y pesaje aplican proyección, pero otros comandos devuelven `obtenerDetalleLote` íntegro. El reporte básico devuelve la meta GMD. Ocultar campos en React no protege la exclusividad comercial del API.

Acción: DTO de salida central por plan, usado en todas las rutas y respuestas; probar que una mutación autorizada en Esencial no revele GMD/cumplimiento avanzados.

LE-13 — Pesos de entrada inferidos y GMD inconsistente entre vistas

Evidencia: lote-service.js:548, protocoloEngorde-service.js:228; R8/R16.

Impacto: un pesaje nueve días posterior al ingreso se muestra como peso de entrada; al agregar otro más cercano cambia retrospectivamente. No existe peso de salida. El consolidado incluye crecimiento anterior a la entrada, mientras el detalle lo excluye. Criterios de GMD de una etapa usan la GMD del lote, no un intervalo propio de etapa. El último peso puede ser antiguo y aun así producir candidatos de venta; estos tampoco filtran por sí solos retiro de carne, que debe revisarse al vender.

Acción: definir medición de ingreso/salida con referencia y cobertura; compartir un cálculo acotado por pertenencia/ciclo/etapa según la métrica publicada. Mostrar antigüedad y no confundir estimación con medición. Conservar una advertencia de retiro en candidatos cuando exista.

LE-14 — Un lote cerrado pierde sus indicadores productivos de población

Evidencia: lote-service.js:278, `obtenerDetalleLote:600`, reporteLotes-controller.js:21, CicloEngorde.js.

Impacto: el cierre termina pertenencias; el detalle solo consulta activas y devuelve población cero, GMD/pesos nulos. Conserva días calculados con inicio/cierre, pero no una GMD final ni ganancia total de la cohorte. Consultar ciclos finalizados sigue construyendo detalle desde el lote actual. Un reporte con filtro temporal filtra inicios de lote, no reconstruye población de aquel período. En interfaz se carga solo el ciclo activo/programado y no hay selector de ciclos terminados.

Acción: cierre productivo con cohortes, intervalos, cobertura, salidas y snapshots finales; consulta histórica explícita. Distinguir reportes «lotes iniciados en período» de «rendimiento durante período». Añadir acceso a ciclos finalizados.

LE-15 — Dependencias de cronograma no garantizan orden ni recalculan por fecha real

Evidencia: protocoloScheduler-service.js:25, `calcularCronograma:39`; protocoloEngorde-service.js:180; R9.

Impacto: validar pasos solo comprueba existencia de referencia, no autorreferencia/ciclos ni unicidad global de IDs. Una dependencia hacia un paso procesado después usa el inicio como fallback. Ejecutar tarde un paso no recalcula fechas de dependientes en engorde. También pueden repetirse IDs entre etapas; se buscan pasos globalmente y se generan claves ciclo+paso.

Acción: IDs únicos en toda plantilla, validación de grafo sin ciclos, orden topológico y reglas para fechas reales. Reprogramar dependientes pendientes de manera atómica y trazable; rechazar ventanas invertidas y fechas inválidas antes de cualquier efecto.

LE-16 — El formulario puede fallar al crear un lote básico

Evidencia: Lotes.js:51, `guardar:86`, formulario de alta; Lote.js:10, lote-service.js:495. R2/R17.

Impacto: «Sin etapa» envía cadena vacía y falla el enum. Seleccionar potrero inicial sin animales devuelve 409 después de guardar el lote; al reintentar puede aparecer código duplicado. La actualización usa `valor || undefined`, por lo que cero se pierde; un campo numérico vaciado y convertido a `undefined` se omite en JSON y puede no limpiar la meta previa.

Acción: normalizar opcionales a `null`/ausencia conforme al contrato; distinguir cero y vacío; permitir incorporar animales en alta o asignar potrero después. El alta debe ser atómica y su resultado inequívoco. Probar los valores iniciales del formulario y la edición que limpia metas.

LE-17 — Roles de interfaz, responsable de paso y manejo de errores desalineados

Evidencia: ProtocolosEngorde.js:32, editor y acciones de cierre; Lotes.js, permisosRoles.js:23, protocoloEngorde-service.js:70.

Impacto: un único `soloLectura` controla operaciones con permisos diferentes. Encargado ve «Plantillas/Versionar» aunque el backend reserva configuración a Administrador. El formulario de iniciar ciclo aparece incluso en solo lectura, con responsables vacíos. Todos los pasos se asignan al responsable general, ignorando el rol guardado en la plantilla. Finalizar/Cancelar no manejan el rechazo de la promesa en esos handlers y no deshabilitan doble clic. Avance no actualiza explícitamente el detalle padre del lote.

Acción: permisos por acción, responsable compatible por paso, cierre con manejo de errores/bloqueo mientras guarda y refresco de paneles relacionados. Mantener ejecución del trabajador por tarea asignada y validar cancelación/asignación actual antes de producir efectos.

LE-18 — Fechas civiles y vencimientos difieren de la zona de la organización

Evidencia: `hoy()` y `fecha()` en ProtocolosEngorde.js:9, fechas en Lotes.js:31; lote-service.js:594; R14.

Impacto: `YYYY-MM-DD` se interpreta a medianoche UTC y su presentación local en Costa Rica muestra día anterior. `toISOString().slice(0,10)` puede proponer el día siguiente por la tarde/noche local. El resumen de tareas del lote calcula vencidas por `fechaProgramada` y medianoche del servidor, sin preferir `fechaLimite` ni la zona organizacional. Puede discrepar del módulo Tareas ya remediado.

Acción: reutilizar utilidades de fechas civiles/instantes y vencimiento ya existentes; usar `Organizacion.zonaHoraria` y fecha límite si está presente. Pruebas alrededor de medianoche, con UTC y Costa Rica, y ejecución de paso con hora real.

LE-19 — Falta contabilidad física/económica integrada del ciclo

Evidencia: SuministroAlimentacion.js, lote-service.js:600, indicesProductivos-service.js:377, sección 5 de este informe.

Impacto: no se puede certificar conversión alimenticia real, costo/kg ganado ni margen completo del lote con las salidas actuales. Las compras y ventas existen y se integran a Finanzas, pero no hay reparto temporal de costos de alimento/sanidad por estancia ni moneda/costo histórico de suministros. Es una capacidad ausente, no pérdida demostrada de dinero.

Acción: definir alcance de producto antes de anunciar esa métrica. Si se desarrolla: costos históricos por suministro y moneda, imputación por animal-días/participación explícita, ganancia para la misma cohorte/período, cobertura de consumo y ajustes por entradas/salidas. Mostrar datos insuficientes antes que inventar una conversión o margen.

LE-20 — Consulta de lista costosa e historia visible truncada

Evidencia: lote-controller.js:25, lote-service.js:600, reporteLotes-controller.js:21.

Impacto: listar todos los lotes calcula detalle completo de cada uno, incluidos historial, eventos y todos los pesajes de integrantes; además repite consultas de cantidad/plan. Se ejecuta incluso antes de ocultar analítica a Esencial. El reporte hace otro enriquecimiento completo por lote. Eventos se limitan a 100 sin cursor o endpoint específico para consultar los anteriores. La búsqueda usa la cadena del usuario como regex sin escape/límite.

Acción: proyección resumida y agregaciones para listado, paginación de lotes y eventos, límites de búsqueda y regex literal. Medir costo y latencia con población representativa; esta auditoría identifica el patrón de consultas, no aporta un benchmark de producción.

Secuencia de remediación y criterios de salida

1. **Cerrar rutas de seguridad:** LE-01/02/03/08. DTOs y alcance en todas las llamadas, incluidos servicios invocados desde protocolos. Validar por API usuarios de fincas distintas y suscripción suspendida antes de cualquier escritura.
2. **Hacer atómicos los comandos:** LE-04/05. Compartir sesiones y claves idempotentes, controlar versión y fallos intermedios. Validar con MongoDB replica set, no solo dobles.
3. **Unificar invariantes operativas:** LE-06/07/09/10/11/21. Ciclos, tareas, estados, membresías, fechas, potrero y bitácora coherentes. Añadir carreras alta de integrantes vs cierre del lote, movimiento vs venta y doble ejecución; el índice único de pertenencia no demuestra por sí solo que esos escenarios sean seguros.
4. **Corregir contrato y experiencia:** LE-12/15/16/17/18/20. Proyecciones por plan, formulario real, cronograma, permisos por acción, fechas y navegación histórica.
5. **Cerrar la analítica y definir economía:** LE-13/14/19. Misma cohorte/período en detalle y consolidado, métricas de cierre reproducibles y especificación explícita de costeo/conversión antes de implementarlos.

La validación de salida debe incluir los tres ejecutores con negocio real (PESAJE/SANIDAD/CAMBIO_RACION), un lote cerrado con tareas, restitución de venta, movimientos entre potreros/fincas y navegación por roles/planes. Las 272 pruebas existentes aprobadas y la compilación correcta **no certifican estos escenarios**.

Índice de reproducciones

Caso	Evidencia reproducida	Hallazgo
R1	Mongoose conserva tenant inyectado y estado cerrado.	LE-01
R2	Etapa vacía del formulario no valida.	LE-16
R3	Query real de plantilla por ID solo aísla organización.	LE-02
R4	Efecto de pesaje y tarea ocurren antes del fallo de ciclo.	LE-04
R5	PARCIAL completa tarea y representa 100 %.	LE-07
R6	Repetir etapa duplica ejecuciones; finalizar cancela obligatorios.	LE-07
R7	Retiro guarda pertenencia antes de fallar animal.	LE-05
R8	Peso posterior usado como entrada.	LE-13
R9	Dependencia circular aceptada y programada.	LE-15
R10	Ejecutor sanitario reenvía campos/animales; esquema admite tenant.	LE-03
R11	PUT devuelve GMD aunque feature sea falsa.	LE-12
R12	Stack de retiro llega al controlador sin gate de vigencia.	LE-08
R13	Cierre no toca tarea/ciclo y permite volver a cambiar fecha.	LE-06/09
R14	Fecha UTC civil se presenta el día anterior en Costa Rica.	LE-18
R15	Versionar no contrasta finca del documento con contexto actual.	LE-02
R16	Detalle 1 kg/día frente a consolidado 5,5 kg/día.	LE-13
R17	Alta con potrero inicial y sin animales falla tras guardar lote.	LE-05/16
R18	Salida reemplaza evento individual de entrada.	LE-11

Estas reproducciones describen el comportamiento defectuoso actual y pueden dejar de pasar al remediarlo. Deben convertirse entonces en pruebas de regresión que exijan rechazo, rollback o resultado correcto; no se incorporaron al patrón de `npm test` como expectativas permanentes del producto.